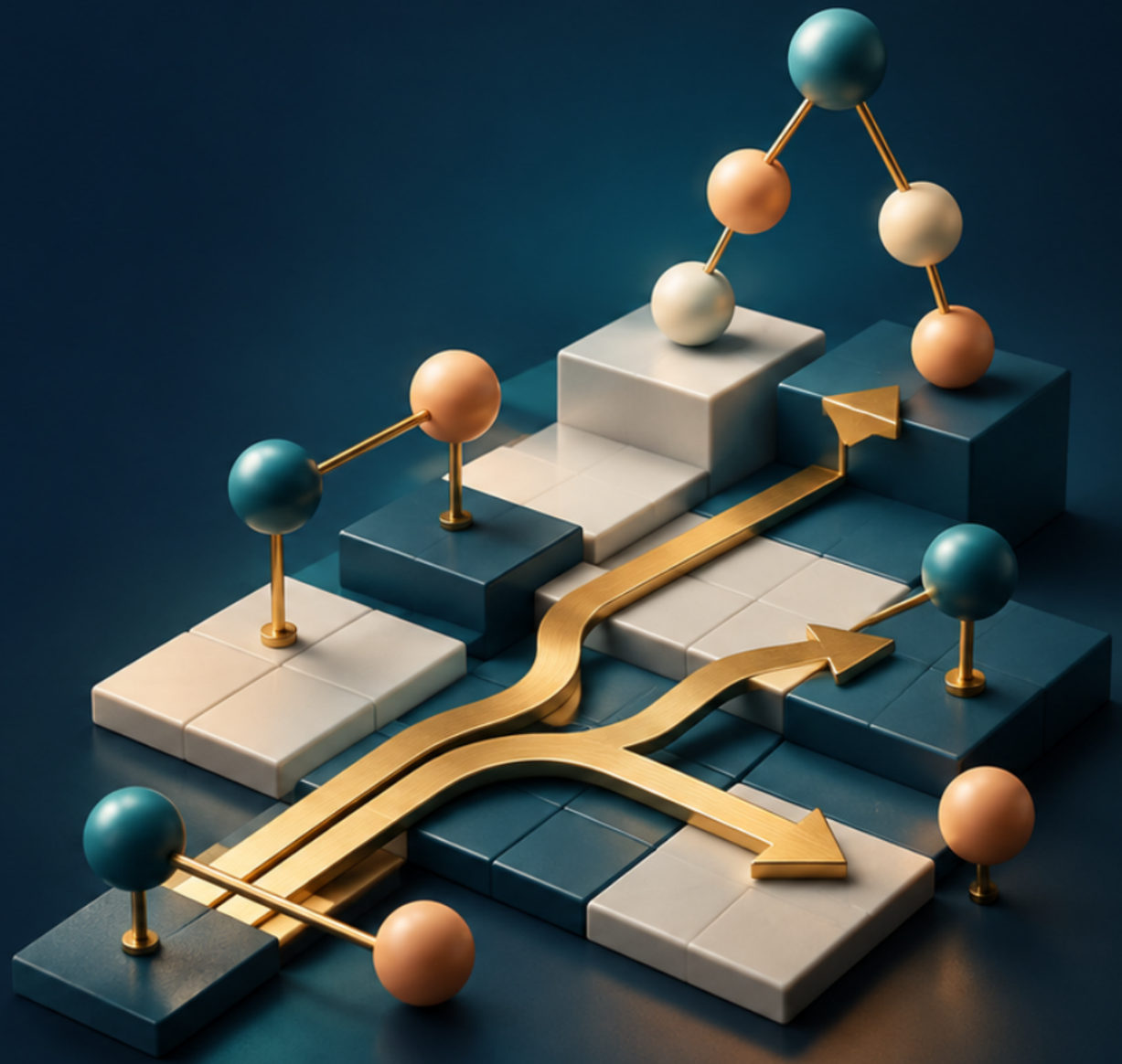
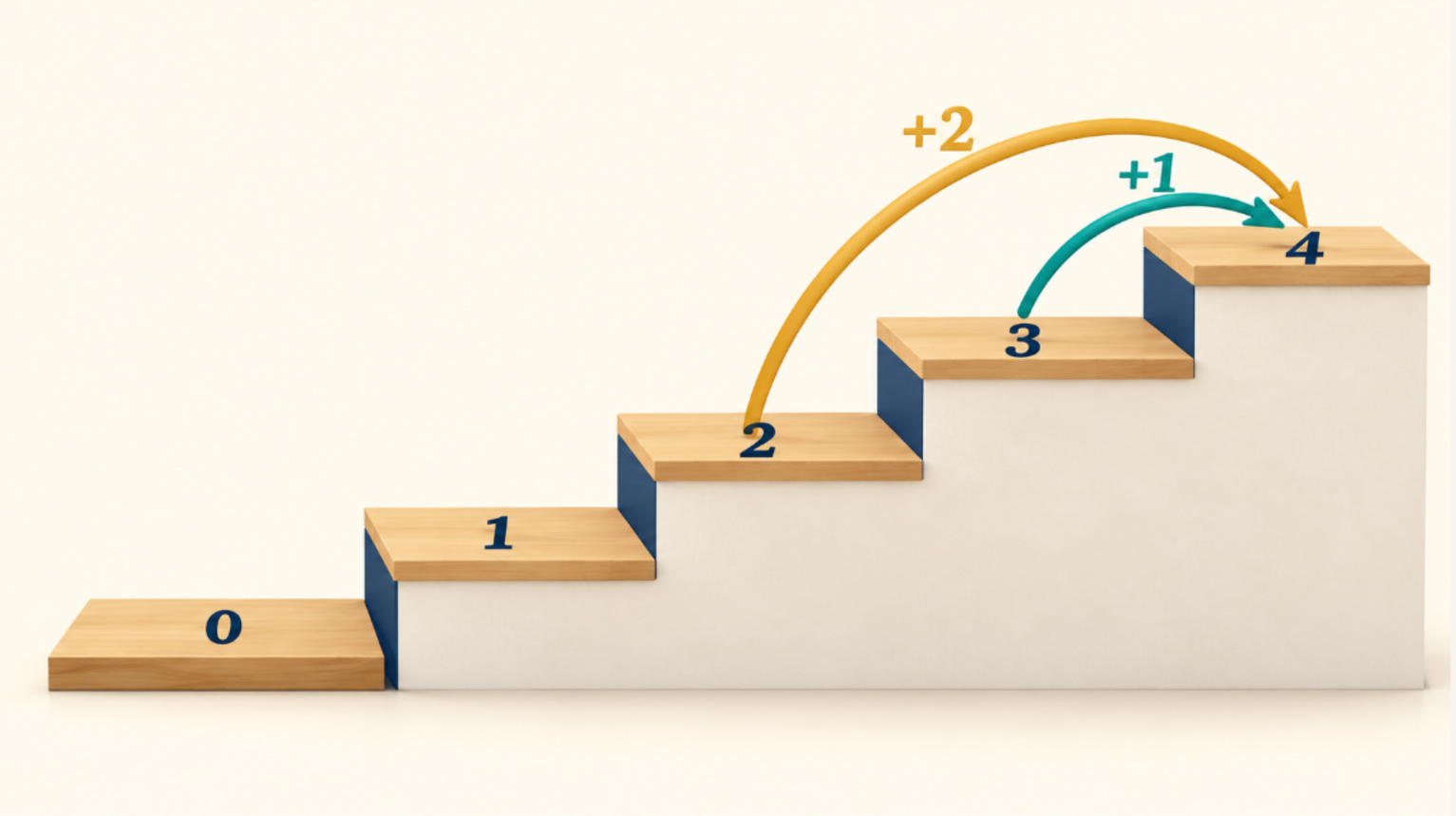
06

동적 계획법

작은 답을 구하고, 다음 계산에 이어 씁니다.



네 칸 계단을 오르는 방법



한 번에 1칸 또는 2칸.
꼭대기까지 가는 순서가
다르면 다른 방법입니다.

네 칸에서는 다섯 가지

오르는 순서	높이 합
1 + 1 + 1 + 1	4칸
1 + 1 + 2	4칸
1 + 2 + 1	4칸
2 + 1 + 1	4칸
2 + 2	4칸

답은 이동 횟수가 아니라 서로 다른 순서의 개수 5입니다.

마지막 걸음으로 두 경우를 나눈다

마지막 이동	직전 위치	그 위치까지 오는 방법
1칸 이동	3칸	1+1+1 / 1+2 / 2+1 : 3가지
2칸 이동	2칸	1+1 / 2 : 2가지

3가지 + 2가지 = 5가지

작은 계단의 답을 더한다

높이	방법 수	앞의 두 답으로 계산
1	1	1칸 한 번
2	2	1+1 또는 2
3	3	2 + 1
4	5	3 + 2
5	8	5 + 3

ways(n) = ways(n-1) + ways(n-2)

탑다운(Top-down)과 바텀업(Bottom-up)

목표는 같은 f(4)=5입니다. 어떤 답부터 구할지가 다릅니다.

방식	시작	작은 답을 얻는 방법
탑다운 Top-down	목표 f(4)부터 요청	**필요한 f(3), f(2)를 재귀 호출로 요청**
바텀업 Bottom-up	f(0)=1, f(1)=1부터 시작	**f(2), f(3), f(4)를 반복문으로 계산**

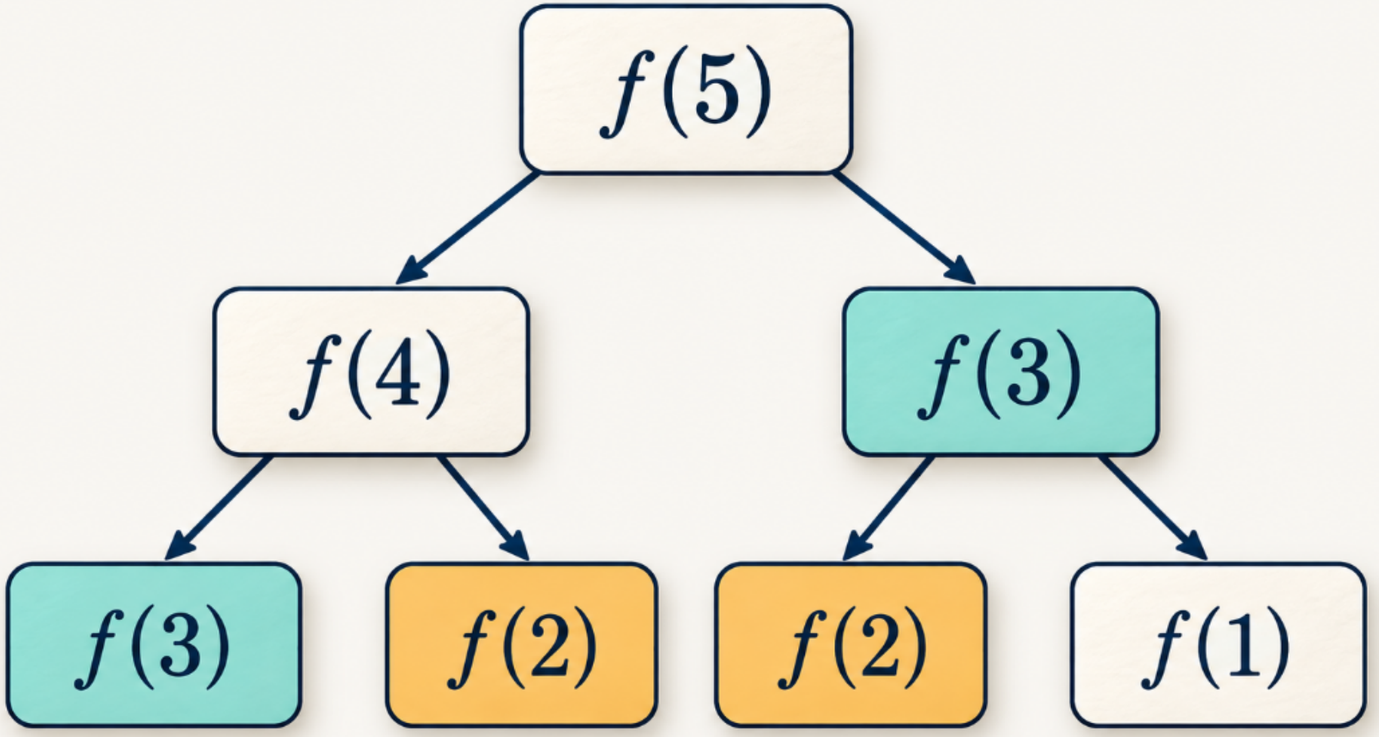
f(n)은 n칸 계단을 오르는 방법 수입니다.
f(0)=1은 아무 이동도 하지 않는 한 가지 방법입니다.
두 방식 모두 f(n)=f(n-1)+f(n-2)를 이용합니다.

그대로 물어보면 같은 계산이 겹친다

계산하려는 답	다시 필요한 답
5칸의 방법 수	4칸의 답 + 3칸의 답
4칸의 방법 수	3칸의 답 + 2칸의 답
3칸의 방법 수	2칸의 답 + 1칸의 답

3칸의 답이 첫째 줄과 둘째 줄에서 다시 필요합니다.

재귀 호출에서 다시 나타나는 같은 질문



f(n)의 뜻

n칸 계단을 오르는 방법 수입니다.

같은 색, 같은 질문

f(3)과 f(2)가 여러 갈래에 다시 나타납니다.

한 번 구한 답

저장해 두면 다음 호출에서 바로 읽습니다.

메모이제이션: 계산한 답 저장과 재사용

요청	저장된 답	할 일
처음 3칸을 계산	아직 없음	**계산해서 3을 저장**
다시 3칸이 필요	3이 있음	**저장한 3을 바로 사용**
4칸을 계산	3칸=3, 2칸=2	**3+2=5를 저장**

계산 결과를 기억하는 방법을 메모이제이션이라고 합니다.

해시맵은 키와 값을 짝지어 저장한다

키: 무엇의 답인가?	값: 그 질문의 답
높이 0	**방법 수 1**
높이 1	**방법 수 1**
높이 2	**방법 수 2**
높이 3	**방법 수 3**
높이 4	**방법 수 5**

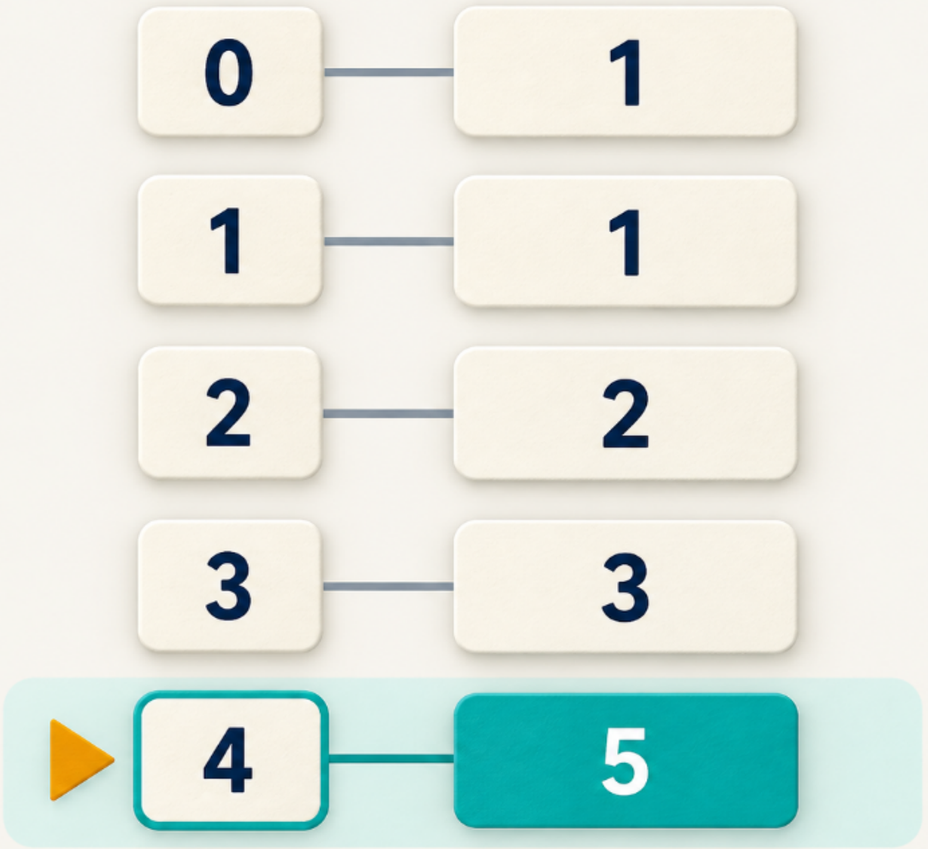
Python에서는 dict, 즉 사전으로 이 표를 만듭니다.

Python 사전에 저장하고 꺼내기

코드	실행한 뒤의 뜻
memo = {0: 1, 1: 1}	0의 답 1, 1의 답 1을 처음에 저장
memo[2] = 2	키 2에 답 2를 저장
2 in memo	키 2가 있으므로 True
memo[2]	키 2에 연결된 값 2를 읽음
4 in memo	아직 키 4가 없으므로 False

in으로 키가 있는지 확인한 다음, 저장된 답을 읽습니다.

높이 4를 찾으면 저장한 답 5가 나온다



왼쪽은 질문

키 4는 높이 4를 가리킵니다.

오른쪽은 저장한 답

값 5는 높이 4까지 오르는 방법 수입니다.

memo[4]를 읽으면

키 4에 연결한 값 5를 가져옵니다.

탑다운(Top-down): 재귀 호출과 메모

```python
def climb_saved(n, memo):
    if n in memo:
        return memo[n]
    memo[n] = climb_saved(n-1, memo) + climb_saved(n-2, memo)
    return memo[n]

memo = {0: 1, 1: 1}
answer = climb_saved(4, memo)
```

필요한 답을 내려가며 찾고, 계산한 결과를 메모합니다.

탑다운 실행 ① 필요한 답을 요청하는 순서

시작 메모: memo={0:1, 1:1}. 먼저 climb_saved(4, memo)를 호출합니다.

순서	요청한 문제	아직 답이 없으면
1	f(4)	**먼저 f(3)을 호출**
2	f(3)	**먼저 f(2)를 호출**
3	f(2)	**f(1)과 f(0)의 저장값 1을 읽음**

목표 f(4)에서 필요한 작은 문제로 내려갑니다.
덧셈 왼쪽의 호출을 먼저 실행합니다.

탑다운 실행 ② 답을 반환하고 메모에 저장

작은 답이 준비되면, 그 답을 기다리던 함수의 계산을 이어갑니다.

처리 위치	읽는 답	실행 결과
f(2)	f(1)=1, f(0)=1	**memo[2]=2 저장**
f(3)	f(2)=2, f(1)=1	**memo[3]=3 저장**
f(4)에서 f(2) 재요청	**이미 저장한 memo[2]**	**값 2를 바로 읽음**
f(4)	f(3)=3, f(2)=2	**memo[4]=5 저장 후 반환**

메모이제이션(Memoization)
계산한 함수의 결과를 저장해, 같은 입력의 재호출에 사용합니다.

바텀업(Bottom-up): 작은 답부터 채우기

높이 i	0	1	2	3	4	5	6
방법 수	1	1	2	3	5	8	**13**

태블레이션: 0칸과 1칸의 답에서 시작해 필요한 높이까지 채웁니다.

바텀업 실행: 작은 칸부터 차례로 채우기

반복문은 i를 2, 3, 4로 바꾸며 진행합니다.

작업	dp[0]	dp[1]	dp[2]	dp[3]	dp[4]
시작값	**1**	**1**	미계산	미계산	미계산
i=2	1	1	**2**	미계산	미계산
i=3	1	1	2	**3**	미계산
i=4	1	1	2	3	**5**

태뷸레이션(Tabulation)
작은 문제의 답을 순서에 맞게 표에 채워 나갑니다.
청록색 칸이 이번 단계에서 새로 채운 칸입니다.

6-2 계단 오르기

높이가 n칸인 계단을 바닥에서 꼭대기까지 오른다. 한 번에 1칸 또는 2칸을 오를 수 있다. 오르는 순서가 다르면 서로 다른 방법으로 센다.

02

꼭대기까지 오르는 서로 다른 방법의 수를 반환하시오.

`climb_stairs(n)`

6-2 입력과 반환값

입력 n

계단 높이. 0~90의 정수

반환값

1칸 또는 2칸씩 올라가는 서로 다른 방법의 개수

처음 두 값

dp[0]=1, dp[1]=1. n=0은 이동하지 않는 방법 한 가지

6-2 이동하지 않음 · 일곱 칸

이동하지 않음

n=0

반환값 1

빈 이동 순서 한 가지를 센다.

일곱 칸

n=7

반환값 21

5칸의 8가지와 6칸의 13가지를 더해 21가지다.

바텀업 구현: 반복문과 배열

```python
def climb_stairs(n):
    dp = [0] * (n+1)
    dp[0] = 1
    if n >= 1:
        dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

시간 O(n), 표 공간 O(n).

C에서 int와 long long을 고르는 이유

저장할 값	실제 크기	사용할 정수형
계단 높이 n	0~90	**int**
46칸의 방법 수	2,971,215,073 (약 29억)	**long long**
32비트 int의 최댓값	2,147,483,647 (약 21억)	**46칸의 답을 담기에는 작음**

입력은 작아도, 계산 결과는 int의 범위를 넘을 수 있습니다.

C 함수의 첫 줄을 읽는 법

long long

결과를 큰 정수형으로 반환합니다.

climb_stairs(int n)

함수 이름은 climb_stairs. 입력 높이를 n이라는 이름으로 받습니다.

{ ... } 와 return

중괄호 안이 함수가 할 일입니다. return이 결과를 돌려주며 끝냅니다.

6-2 C

```
1  long long climb_stairs(int n) {
2      long long dp[91] = {0};
3      dp[0] = 1;
4      if (n >= 1) dp[1] = 1;
5      for (int i=2; i<=n; ++i) {
6          dp[i] = dp[i-1] + dp[i-2];
7      }
8      return dp[n];
9  }
```

시간 O(n), 표 공간 O(n).

C의 기호를 말로 읽어 보기

표현	말로 읽으면
dp[0] = 1;	0번 칸에 1을 저장한다.
n >= 1	n이 1 이상인가 검사한다.
int i=2; i<=n; ++i	2번부터 시작해서 n번까지, 번호를 하나씩 늘린다.
dp[i-1] + dp[i-2]	한 칸 앞의 값과 두 칸 앞의 값을 더한다.
return dp[n];	n번 칸의 값을 답으로 돌려주고 끝낸다.

등호 = 는 저장, == 는 같은지 비교할 때 씁니다.

탑다운과 바텀업의 비교

방식	진행 방향	답을 저장하는 때
탑다운 + 메모	큰 질문에서 필요한 작은 질문으로	**필요한 답을 계산한 뒤 저장**
바텀업 + 표	0칸, 1칸부터 원하는 높이까지	**작은 답부터 순서대로 저장**

이 수업의 네 코딩 문제는 반복문으로 푸는 바텀업을 사용합니다.

메모이제이션과 태뷸레이션

구분	탑다운에서의 저장	바텀업에서의 저장
이름	메모이제이션	**태뷸레이션**
저장 시점	호출한 문제의 답이 나올 때	**각 칸의 계산을 마칠 때**
재사용하는 때	같은 입력으로 다시 호출할 때	**다음 칸을 계산할 때**
저장 자료구조	배열 또는 사전	**배열 또는 사전**

계산 순서와 저장 자료구조를 구분합니다.
탑다운에서도 배열을, 바텀업에서도 사전을 사용할 수 있습니다.

탑다운·바텀업의 선택과 비용

계단 예제의 비용	탑다운 + 메모	바텀업 + 배열
계산 시간	O(n)	**O(n)**
답 저장 공간	O(n)	**O(n)**
추가 재귀 호출 공간	최대 O(n)	**없음**

필요한 상태가 드문 문제는 탑다운을 고려합니다.
정한 순서로 대부분의 칸을 채우는 문제는 바텀업이 편합니다.

6-3 연속 부분합

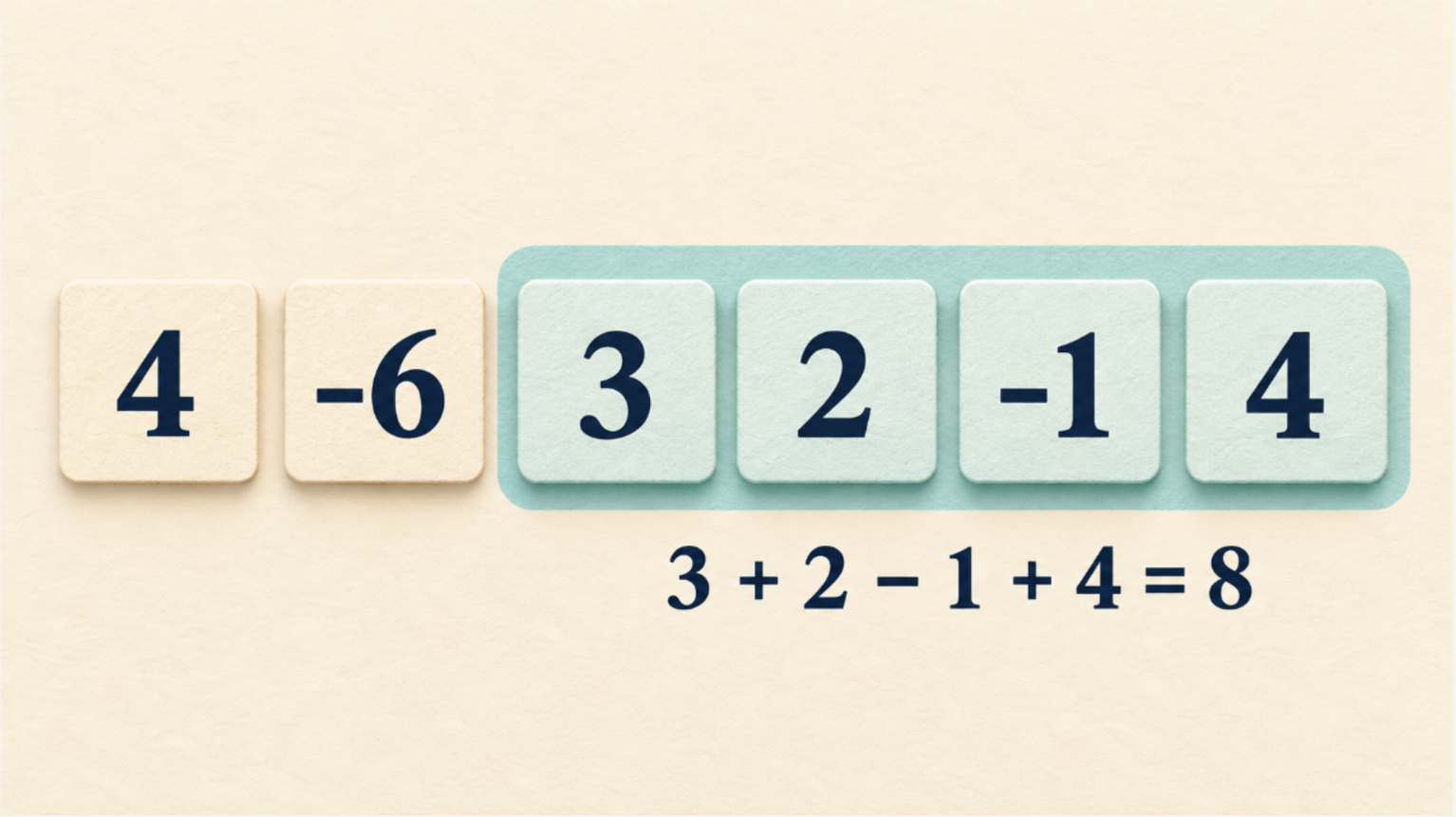
정수 배열 a에서 하나 이상의 원소가 연속해서 이어진 구간 하나를 고른다. 구간 중간의 원소를 건너뛸 수 없다.

03

고른 구간의 원소 합 중 가장 큰 값을 반환하시오.

```
max_subarray(a)
```

연속된 구간 하나를 고른다



3부터 마지막 4까지 모두 포함합니다.
합은 3+2-1+4=8입니다.

이번 수에서 끝나는 구간만 먼저 본다

끝을 세 번째 수 3으로 고정	구간 합
[4, -6, 3]	1
[-6, 3]	-3
[3]	3

세 번째 수에서 끝나는 구간의 최대 합은 3입니다.

이어 붙일까, 새로 시작할까

바로 앞에서 끝난 최대 합

4 + (-6) = -2

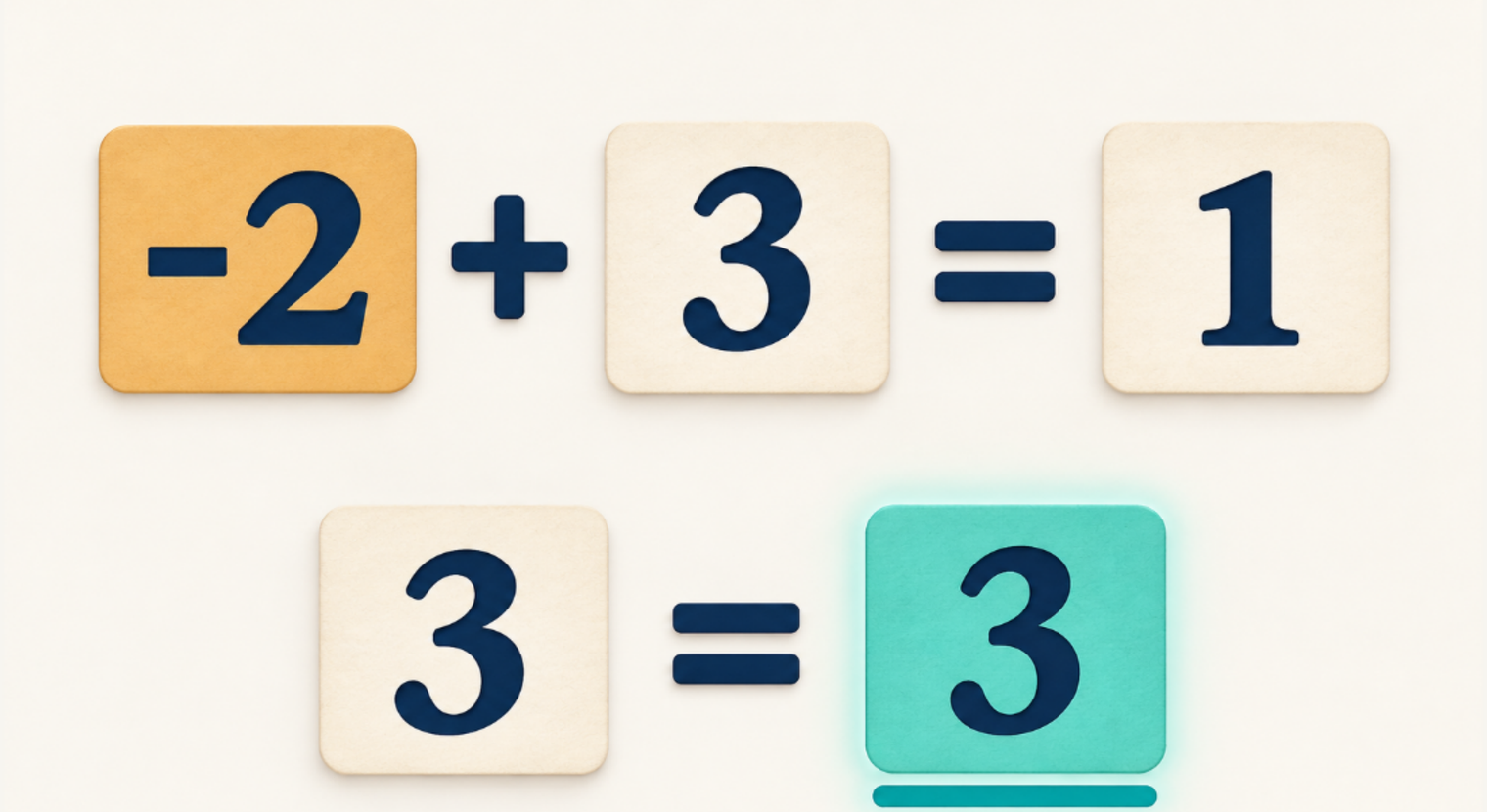
이번 수 3을 이어 붙이면

-2 + 3 = 1

이번 수부터 새로 시작하면

3 하나의 합 = 3. 더 큰 3을 선택

앞의 합이 음수일 때 새로 시작하는 경우



이어 붙이기

앞의 합 -2에 3을 붙이면 1입니다.

이번 수에서 시작

3 하나로 새 구간을 만들면 합은 3입니다.

현재 칸에 저장

둘 중 큰 3을 ending에 둡니다.

지금까지의 최고 합은 따로 기억한다

ending

이번 위치에서 반드시 끝나는 구간의 최대 합

best

지금까지 본 모든 구간의 최대 합

세 번째 수 3까지 읽었을 때

ending=3, best=4. 맨 앞의 [4]가 아직 최고

여섯 수를 끝까지 따라가 보기

이번 수	이전 ending에 붙임	새로 시작	새 ending	새 best
4	시작	4	4	**4**
-6	4-6=-2	-6	-2	**4**
3	-2+3=1	3	3	**4**
2	3+2=5	2	5	**5**
-1	5-1=4	-1	4	**5**
4	4+4=8	4	8	**8**

마지막 ending과 기존 best를 비교한 뒤, best=8을 반환합니다.

6-3 입력과 예외 조건

입력 배열 a

원소 1~10,000개. 각 값은 -1,000,000~1,000,000

반환값

하나 이상 연속해서 고른 원소들의 최대 합

처음 값

ending과 best를 첫 원소 a[0]으로 시작합니다.

6-3 모두 음수 · 원소 한 개

모두 음수

```
a=[-8,-3,-5]
```

반환값 -3

하나는 골라야 하므로 가장 큰 -3을 고른다.

원소 한 개

```
a=[-7]
```

반환값 -7

유일한 구간 [-7]의 합 -7을 반환한다.

6-3 Python

```python
def max_subarray(a):
    ending = a[0]
    best = a[0]
    for i in range(1, len(a)):
        ending = max(a[i], ending + a[i])
        best = max(best, ending)
    return best
```

시간 O(n), 추가 공간 O(1).

6-3 C 함수와 입력 배열

long long max_subarray(...)
함수가 돌려줄 최대 합은 큰 정수입니다.

const int a[], int n
a는 입력 원소들, n은 원소 수입니다. a를 읽기만 합니다.

a[0]
첫 번째 원소. 원소가 최소 하나이므로 시작값으로 쓸 수 있습니다.

6-3 C

```
1  long long max_subarray(const int a[], int n) {
2      long long ending=a[0], best=a[0];
3      for (int i=1; i<n; ++i) {
4          long long extend=ending+a[i];
5          if (extend>a[i]) ending=extend;
6          else ending=a[i];
7          if (ending>best) best=ending;
8      }
9      return best;
10 }
```

시간 O(n), 추가 공간 O(1).

모두 음수일 때 0으로 시작하면 틀린다

읽은 값	ending	올바른 best
-8	-8	**-8**
-3	-3	**-3**
-5	-5	**-3**

처음 값을 a[0]으로 잡아야 실제로 고른 구간의 합이 남습니다.

다음 계산에 필요한 정보를 남긴다

계단에서 기억한 답

해당 높이까지 오는 방법 수. 앞의 두 답을 더합니다.

연속 부분합에서 기억한 답

이번 위치에서 끝나는 최대 합 ending, 지금까지의 최고 합 best

DP를 설계할 때 먼저 할 일

저장한 숫자가 무엇을 뜻하는지 한 문장으로 정합니다.

두 문제에서 저장하는 답

문제	저장하는 답	다음 계산
계단 오르기	높이 i까지 오는 방법 수	앞의 두 방법 수를 더함
연속 부분합	현재 위치에서 끝나는 최대 합	이어 붙이기와 새로 시작 중 큰 합

계단에서는 방법 수를 더하고, 연속 부분합에서는 더 큰 합을 고릅니다.

반복 계산을 줄이기 전에 확인할 것

작은 답을 정확히 정의

무엇을 입력으로 구분하고, 어떤 값을 답으로 저장할지 정합니다.

다음 계산에 충분한 정보

연속 부분합은 best만으로 부족해 ending도 기억했습니다.

준비된 답을 읽는 순서

필요한 이전 답을 먼저 구한 뒤 현재 답을 계산합니다.

계단 오르기와 연속 부분합 연습

번호	난이도	문제	반환할 값
6-2	쉬움	계단 오르기	오르는 방법 수
6-3	기본	연속 부분합	연속 구간의 최대 합

사이트의 6-2와 6-3을 풀고, 각 문제의 시작 조건도 확인해 보세요.